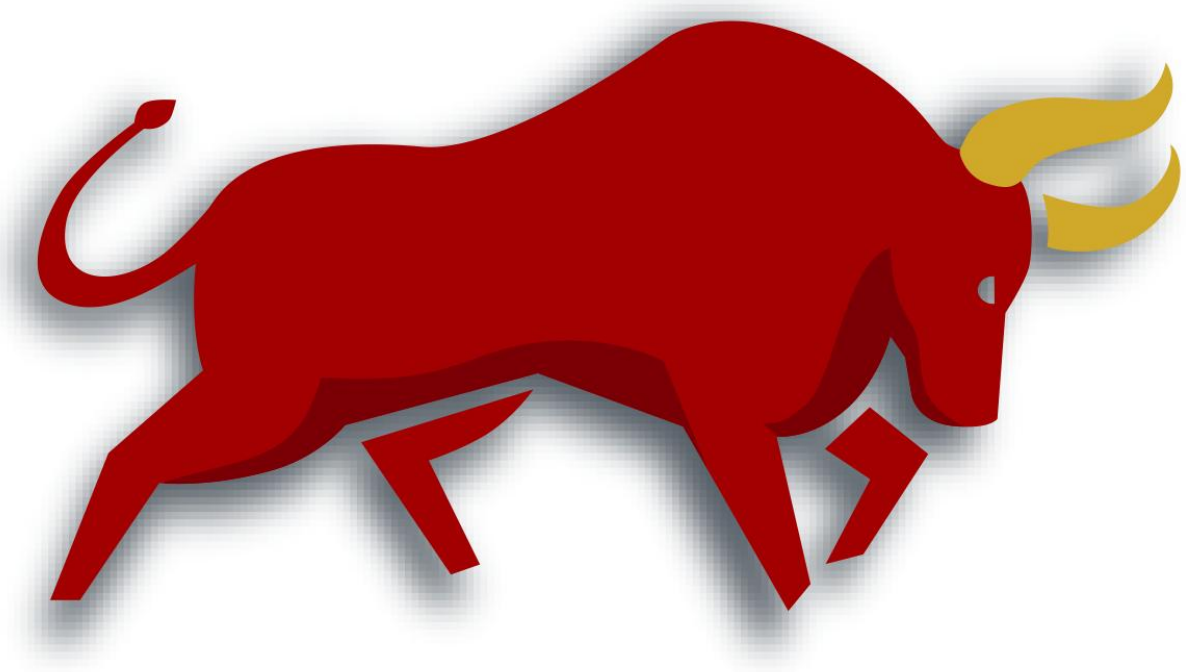
FDT Big Data Centre



FDT Scoring Platform

2015-2016

Written by Pengfei Liu (pengfei.liu@hkfdt.com)

Financial Big Data Centre

Financial Data Technologies Limited.

Building 12W, Hong Kong Science and Technology Park

Hong Kong

Overview

This project aims to ....

Goals

1. Create each user a profile
2. Grade each user with a FDT score and other types of scores
3. Visualize each user's profile

System Architecture

The system architecture is shown in Figure 1:

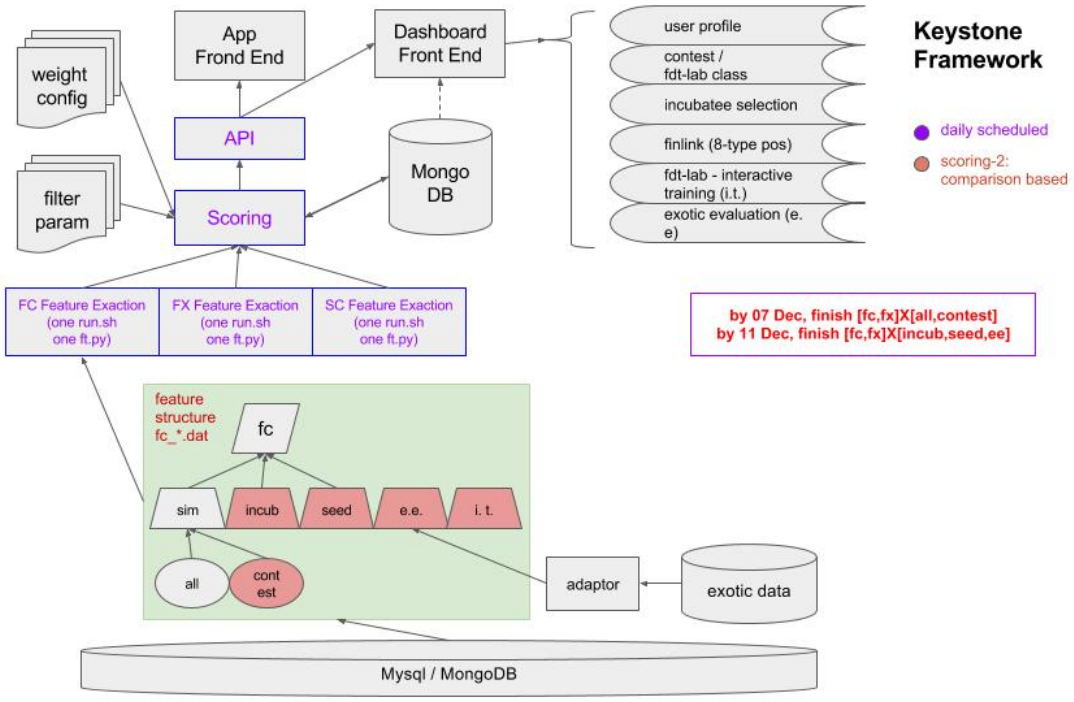


Figure 1: The Architecture of the User-Profile-Modeling Project (By Qifeng)

Note: we support different types of products: fx, fc and sc and different types of data sources. In the bottom tree structure, **each branch from a product (e.g., fc) to the leaf** defines a unique data source. (**!!Important!!**) We maintain exactly one copy of feature repository (there are **19** features for now), where each feature extractor will extract features for each data source.

Challenges

The "**FDT Scoring Platform**" needs to handle the following challenges:

We solved these challenges by the following **Three-One** design strategies:

1. **One single feature repository** shared by all data sources and all applications. Features can be selected based on data sources and applications.
2. **One tree structure** to represent multiple data sources, where each branch uniquely defines a data source.
3. **One weight schema database** to store various weights schemas, such 8 weights files for 8 types of financial candidates in FinLink.

System Implementation

The framework for the "**FDT Scoring Platform**" project (risk, preference and fdt scores) has been established.

Overview

The framework for the "**FDT Scoring Platform**" project (risk, preference and fdt scores) has been established. It contains four major components:

1. Feature engineering scripts
2. Script execution engine
3. MongoDB for saving features
4. APIs for other applications

There are two background jobs running:

1) **feature-extraction**: this is the script-execution engine which runs the scripts in the feature-extractors folder (/opt/user-profile/features). This job is daily scheduled.
2) **compute-scores**: this is the feature-rating engine which rates each feature by an algorithm like grading students in a course. The score range is [0, 100]. (MongoDB)

Feature Engineering

We design a lot of features for calculating the FDT score. Each feature is a folder with necessary scripts and must be located in the folder "/opt/user-profile/features". The detailed documents for current features are available at:

https://docs.google.com/a/hkfdt.com/document/d/1nyrBkxmSzY37K98gqsWibL_k3L5MUNV4P42fm0wzWh8/edit?usp=sharing

Steps to Add a New Feature

To ensure the project quality, the following code conventions for adding new features to the framework:

1. Create the main script "run.sh" which will create the file "data_source.dat" (e.g., **fx_sim_all.dat**) (Note: the first line of run.sh should be: "#!/bin/sh", which is a comment on shell location.)
2. Create a folder "dataset" which contains the files obtained from MySQL or other data sources
3. Create an optional "debug" folder if you want to write some debug files (mkdir -p debug)
4. Write a "README.txt" file to describe your feature
5. (Optional) Create a file "test.txt" to contain the top 10 rows of your "feature.dat" output
6. Copy your features to the folder /opt/user-profile/features/ (python and shell scripts, no generated data files or temp files, which should be "run.sh", "*.py", and "README.txt")

Note: **To support multiple data sources, we revises step one as follows:**

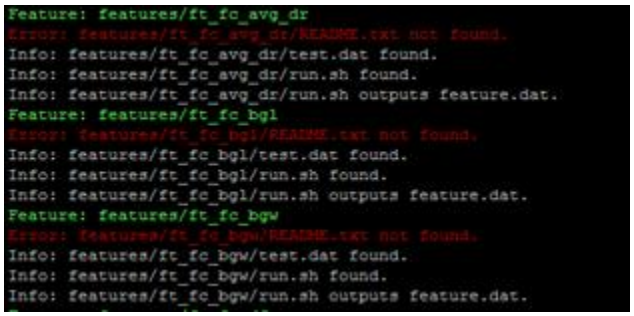
!!Important!!

Create the main script "run.sh" which will create one .dat file for each data source, each data source should have exactly one .dat file, such as **fx_sim_all.dat** or **fc_sim_all.dat**. You can write multiple python scripts for fx, fc and sc or reuse the same script file for fx, fc and sc.

!!Important!!

The code conventions can be checked automatically. An example command is like this:

```
[pengfei@iZ23bz3npd1Z user-profile-daemon]$ make check-code
```



Support External Data Sources

We extend the project further to support multiple external data sources. This is challenging due to various data sources, different weighting schemes, various scoring strategies. The implementation is as follows:

1) Create a new Mongo collection “externalUser” to save all users from external data sources;
2) Similar to fx, fc and sc, all external data sources are prefixed with “ex”;
3) Each external data source has a unique key, e.g., ex_source_1, which will be used as feature filename, weights schema filename and so on;
4) Specific APIs will be developed for externalUser;

MongoDB Design

We design a MongoDB named “UserProfile”, which consists of the collections of “user” and “extUser” for now.

Weights Schemes

Each application will have a specific weight scheme. All weights schemes are located in the folder “weights”.

Front End Visualization

We mainly use radar charts to visualize the scores for each user. The demo page can be found at http://121.43.73.64/#/index/userScore.

System Operation

Source Code

Currently, the source code is located at **/opt/user-profile-daemon** on the aliyun server (10.139.96.26).

Folder	Comments
/opt/user-profile-daemon/	Source code
/var/log/user-profile-daemon/	Log files
/opt/user-profile/features/	Feature files

Running Project

When you are in the folder of /opt/user-profile-daemon/, you can easily run the project using the commands below:

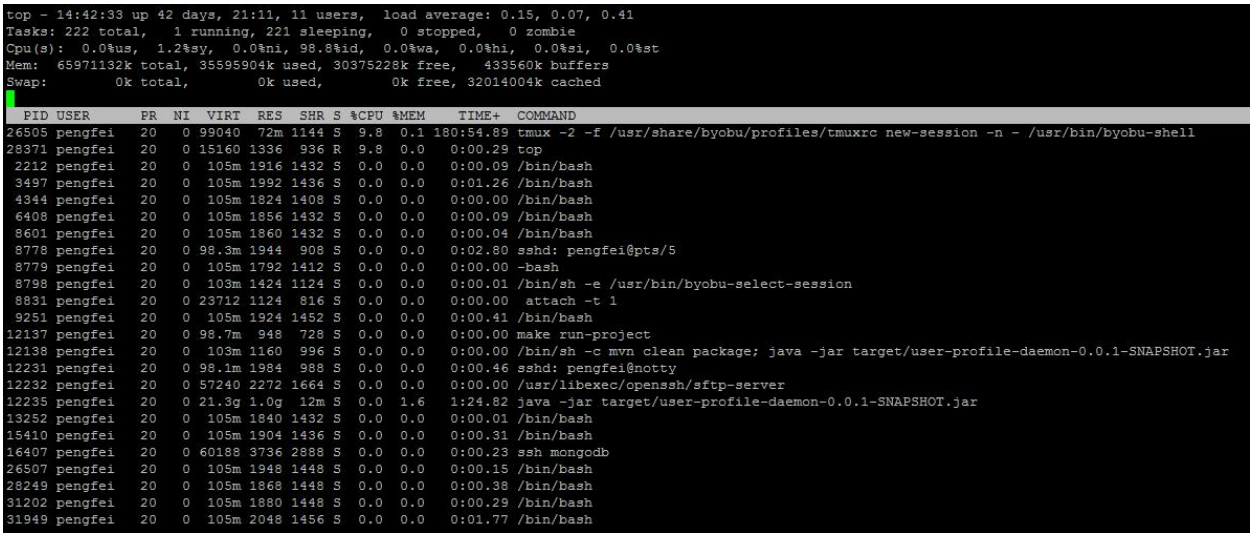
(1) Compile the project again only if you changed the source code

[pengfei@iZ23bz3npd1Z user-profile-daemon]$ **make compile-project**

(2) Run the project

```
[pengfei@iZ23bz3npd1Z user-profile-daemon]$ make run-project
```

Please make sure there is only one process running for user-profile-daemon. Kill the old process if you want to run a new user-profile-daemon, as shown in the below figure by the "top" command. The daemon is a Java process named "user-profile-daemon-0.0.1-SNAPSHOT.jar"



Scheduling Jobs on-demand

The platform now supports scheduling the jobs on demand:

(1) Invoke all run.sh

curl http://10.139.96.26:8099/api/scores?job=compute

This job may take around **10 minutes** to finish.

(2) Save features to mongodb

curl http://10.139.96.26:8099/api/scores?job=save

You can run these commands using your own account on aliyun.

Tuning Weights

The weights of features need to be tuned from time to time. We maintain a separate weights file for the project, which can be changed on demand.

Compared with the old properties files based solution, we have developed a new XML based version, with the following advantages:

1) Weights files are now stored in more structured XML files;
2) Two weights files are provided: trader.xml (effective by default) and banker.xml.
3) The effective weights file can be changed in application.properties, e.g., to be banker.xml.
4) A new group of features can be added directly to the XML file, e.g., trader.xml. (Please double check the feature names to be the same as /opt/user-profile/features).

Note that, the project needs to be restarted for any changes to the configurations files.

We currently support both “banker.xml” (for banker scenario) and “trader.xml” (for trader scenario). An example command is like this:

[pengfei@iZ23bz3npd1Z user-profile-daemon]$ **vim src/main/resources/banker.xml**

Or

[pengfei@iZ23bz3npd1Z user-profile-daemon]$ **vim src/main/resources/trader.xml**

We need to specify which file to use in “application.properties”.

```
[pengfei@iZ23bz3npd1Z user-profile-daemon]$ vim src/main/resources/application.properties

server.port = 8098
weights.file = trader.xml
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<features>
	<consistency>

<ft_fc_std_op_dr>0.5</ft_fc_std_op_dr>

<ft_fc_std_op_qty>0.5</ft_fc_std_op_qty>
	</consistency>
	<activity>
		<ft_login_cnt>0.4</ft_login_cnt>
		<ft_trade_cnt>0.6</ft_trade_cnt>
	</activity>
	<profitability>
		<ft_fc_bgw>0.2</ft_fc_bgw>
		<ft_up>0.4</ft_up>
		<ft_streak>0.4</ft_streak>
	</profitability>
	<riskCtrl>

<ft_avg_ngt_op>0.15</ft_avg_ngt_op>

<ft_avg_max_op>0.15</ft_avg_max_op>
```

Any group of weights can be changed, but please **make sure the feature names are correct** and **restart** the project after changing the weights or updating the application.properties file.

Support FinLink

We extend the framework to support FinLink. We have developed a preliminary demo to select 8 types of financial candidates of China Merchants Bank (CMB).

Type	Characteristics	Weight File	Comment
banker_trader_steady	short op duration & good risk control	banker_trader_steady.xml	稳健交易型
banker_trader_active	short op duration & good profitability	banker_trader_active.xml	积极交易型
banker_trader_balanced	short op duration & good balance	banker_trader_balanced.xml	平衡交易型
banker_investor_steady	long op duration & good risk control	banker_investor_steady.xml	稳健投资型
banker_investor_active	long op duration & good profitability	banker_investor_active.xml	积极投资型
banker_investor_balanced	long op duration & good balance	banker_investor_balanced.xml	平衡投资型
banker_risk_control	low risk preference good profitability	banker_risk_control.xml	风险控制型
banker_marketing	good activity good usage	banker_marketing.xml	市场营销型